

California State Polytechnic University, Pomona
Autonomous Vehicle Laboratory
IGVC 2026 Design Report

ROBOT NAME

I hereby certify that the development of the vehicle, *ROBOT NAME*, as described in this report, is equivalent to the work involved in a senior design course.

Submitted May XX, 2026

Software	Mechanical	Electrical
Name	Name (Captain)	Name
Name	Name	Name
Name	Name	Name
Name		

0. Contents

1	Design Process, Team Identification and Organization	2
1.1	Introduction	2
1.2	Team Organization	2
1.3	Design Process	2
2	System Architecture	4
2.1	Mechanical and Electrical Architecture	4
2.2	Robot Safety	4
2.3	Software Architecture	4
2.4	Robot Communication Bus	5
2.5	Bill of Materials	5
3	Effective Innovations	6
3.1	Innovation 1: TODO	6
3.2	Innovation 2: TODO	6
3.3	Custom CAN Firmware for SparkMAX Control	6
4	Mechanical Design	7
4.1	Overview	7
4.2	Drivetrain	7
4.3	Frame and Sensor Mast	7
4.4	Weatherproofing	7
5	Electronic and Power Design	8
5.1	Overview	8
5.2	Power Distribution	8
5.3	Motor Control	8
5.4	Safety Lights	9
5.5	Emergency Stop and Safety	9
6	Software Systems	11
6.1	Overview	11
6.2	Image Transformation	11
6.3	Navigation	11
6.4	Localization	11
6.5	Drive Control	12
6.6	Configuration and GUI	12
6.7	State Machine	12
6.8	Logging	12
7	Cyber Security Analysis	13
7.1	NIST RMF Process	13
7.2	Vehicle Threats	13

7.3	Cyber Controls and Implementation	14
8	Vehicle Analysis	15
8.1	Lessons Learned	15
8.2	Failure Points and Resolutions	15
8.3	Safety, Reliability, and Durability	15
8.4	Performance Metrics	16
8.5	Software Testing, Bug Tracking, and Version Control	16
8.6	Simulation	16
9	Unique AutoNav and Self Drive Software	17
10	Performance Assessment	18

1. Design Process, Team Identification and Organization

1.1 Introduction

The Autonomous Vehicle Laboratory (AVL), representing California State Polytechnic University, Pomona (Cal Poly Pomona) and the College of Engineering, proudly presents *ROBOT NAME*, our entry for the 2026 Intelligent Ground Vehicle Competition (IGVC) in the AutoNav category. AVL is a multidisciplinary student organization within the College of Engineering, with a dedicated IGVC team of XX students. The team is advised by ADVISOR NAME(S), faculty member(s) at Cal Poly Pomona.

Cal Poly Pomona’s learn-by-doing philosophy is at the core of AVL’s approach. Every component of the vehicle—from mechanical design to embedded firmware to autonomous navigation software—is designed, built, and tested by students. This is the team’s first entry in the IGVC competition.

1.2 Team Organization

The IGVC team is organized into three specialized subteams: electrical, mechanical, and software. A team captain serves as the primary point of contact with AVL leadership and faculty advisors, providing regular updates on progress and coordinating across subteams.

- **Mechanical:** Chassis design, drivetrain, sensor mounting, weatherproofing
- **Electrical:** Power distribution, motor controllers, PCB design, CAN bus, e-stop
- **Software:** Perception, navigation, localization, firmware, GUI

To promote communication and prevent silos, the team holds weekly full-team meetings in addition to subteam work sessions. All design decisions are documented and version-controlled.

1.3 Design Process

To develop *ROBOT NAME*, the team implemented an iterative design process that focuses on a cycle of researching, designing, prototyping, and testing.

1. **Define** — Establish system-level requirements from competition rules.
2. **Research** — Survey existing solutions, reference designs, and prior IGVC reports.
3. **Design** — Create detailed CAD models, schematics, and software architecture.
4. **Prototype** — Build rapid prototypes (3D-printed parts, breadboard circuits, simulation).
5. **Test** — Validate each subsystem independently, then integrate.
6. **Implement** — Finalize manufacturing and deploy to the competition vehicle.

The team began by analyzing design reports from prior IGVC winners and performing a full re-read of the 2026 competition rules to derive a list of rule-based requirements. From these, the team established high-level design goals. With the system-level requirements defined, subteams proceeded to research, design, build, and test the necessary components. As each

subsystem was completed, integration testing ensured that all components worked cohesively and that the robot met its overall requirements.

2. System Architecture

2.1 Mechanical and Electrical Architecture

At a high level, *ROBOT NAME* consists of several independent subsystems: chassis, drivetrain, vision, sensors, and electronics.

Table 1: Major subsystems overview.

Subsystem	Description
Chassis	TODO: Material, dimensions, design philosophy
Drivetrain	Differential drive with 2× REV SparkMAX controllers and NEO/550 motors
Compute	TODO: Intel NUC / Jetson / Raspberry Pi
Microcontroller	Teensy 4.1 — CAN bridge between main computer and motor controllers
Sensors	TODO: Camera(s), GPS/IMU, ultrasonics/LiDAR
Power	TODO: Battery type, voltage, capacity

2.2 Robot Safety

ROBOT NAME was designed with safety as a fundamental principle. Power is delivered to the drive motors only when the following conditions are simultaneously met:

1. The **remote emergency stop** is inactive (wireless e-stop not triggered).
2. The **physical emergency stop** button on the vehicle is not pressed.
3. The **software emergency stop** has not been asserted.

If any one of these is triggered, motor power is immediately cut. To prevent accidental re-engagement, each of the three signals is also monitored redundantly in software. As a result, the robot will not send any non-zero command to the motors unless all safety interlocks are satisfied.

2.3 Software Architecture

The software is split into four major subsystems: sensors, perception, localization, and navigation.

- **Sensors** — Camera capture, GPS/IMU reading, motor encoder feedback
- **Perception** — Image processing, obstacle detection, lane detection
- **Localization** — Sensor fusion (wheel odometry + GPS) for position estimation
- **Navigation** — Path planning and motor command generation

Organizing the code in this modular way allows for easy testing of each subsystem independently as well as making testing different algorithms or parameters quick and easy.

2.4 Robot Communication Bus

Data is communicated between devices using the **CAN bus** protocol, a robust standard widely used in the automotive industry. The Teensy 4.1 microcontroller serves as the CAN bridge between the main onboard computer and the REV SparkMAX motor controllers.

The CAN bus operates at **1 Mbit/s** using 29-bit extended frames. A CAN transceiver (SN65HVD230) converts the Teensy’s logic-level signals to differential CAN_H / CAN_L. The bus is terminated with $120\,\Omega$ resistors at each end.

2.5 Bill of Materials

The total cost to construct *ROBOT NAME* was approximately \$X,XXX.

Table 2: Major COTS components.

Major COTS Item	Qty	Market Cost (USD)	Team Cost (USD)
TODO: Main Computer	1	\$XXX.00	\$XXX.00
TODO: Camera(s)	1	\$XXX.00	\$XXX.00
TODO: Battery	1	\$XX.00	\$XX.00
TODO: GPS/IMU	1	\$X,XXX.00	\$XXX.00
REV SparkMAX + NEO Motor	2	\$XXX.00	\$XXX.00
Teensy 4.1	1	\$30.00	\$30.00
CAN Transceiver (SN65HVD230)	1	\$5.00	\$5.00

3. Effective Innovations

3.1 Innovation 1: TODO

TODO: Describe your team's primary mechanical or drivetrain innovation.

3.2 Innovation 2: TODO

TODO: Describe your team's primary software or perception innovation.

3.3 Custom CAN Firmware for SparkMAX Control

The team developed custom firmware for the Teensy 4.1 microcontroller that communicates directly with REV SparkMAX motor controllers over CAN bus. This eliminates the need for a roboRIO or additional middleware, reducing cost, latency, and complexity. The firmware handles:

- Non-RIO heartbeat broadcast to keep SparkMAXes enabled
- Percent-output, velocity, and position command modes
- Periodic encoder feedback parsing for odometry
- Serial interface for integration with the main onboard computer

4. Mechanical Design

4.1 Overview

The mechanical design of *ROBOT NAME* is focused on simplicity, reliability, ease of access to electronics, weatherproofing, and modularity. The vehicle uses a differential drive configuration with two independently driven wheels and one or two passive caster wheel(s) for stability.

4.2 Drivetrain

ROBOT NAME features a differential drive system powered by two REV SparkMAX motor controllers, each driving a REV NEO brushless motor. The differential drive configuration was chosen for its simplicity, reliability, and proven performance in outdoor environments.

Table 3: Drivetrain specifications.

Parameter	Value
Drive type	Differential
Motors	2× REV NEO
Motor controllers	2× REV SparkMAX
Gear ratio	TODO
Wheel diameter	TODO
Wheel type	TODO
Designed top speed	TODO
Track width	TODO

Each motor is connected to its wheel through a gearbox/belt/chain drive. The gear ratio was selected so that the peak of the motor power curve falls at typical IGVC operating speeds.

4.3 Frame and Sensor Mast

The frame is constructed from TODO: material (e.g., 80/20 aluminum extrusion, welded aluminum, steel tube). A deliberate effort was made to design for ease of access to internal electronics and for modularity so that subsystems can be quickly swapped or repaired.

A sensor mast is mounted at the center/front of the robot to elevate the camera(s) and GPS antenna. Maximizing the height of the cameras provides a wider field of view and allows the image to be projected onto a 2D plane with minimal distortion.

4.4 Weatherproofing

ROBOT NAME features acrylic panels / polycarbonate enclosure for protection from rain and sun. Weatherstripping and gaskets protect sensitive electronic components. All cable pass-throughs are sealed to prevent water ingress.

5. Electronic and Power Design

5.1 Overview

The electronic system of *ROBOT NAME* is designed for modularity and repairability. The architecture features the following key components:

- **Main computer** — TODO: describe
- **Teensy 4.1** — CAN bridge handling motor control and encoder feedback
- **CAN transceiver** — SN65HVD230 converting logic-level to differential CAN
- **2× REV SparkMAX** — Brushless motor controllers with built-in PID
- **E-stop system** — Hardware relay controlled by physical + wireless + software interlocks
- **Safety lights** — TODO: RGB LED strip / indicator lights

5.2 Power Distribution

The vehicle uses a TODO: 12V XXAh battery to power both the electronics and the drive-train.

Table 4: Power budget.

Component	Voltage (V)	Peak Current (A)	Avg Current (A)
Main computer	TODO	TODO	TODO
Teensy 4.1	5 (via USB)	0.1	0.05
SparkMAX + motors	12	TODO	TODO
Sensors / cameras	TODO	TODO	TODO
Safety lights	TODO	TODO	TODO
Total	—	TODO	TODO

5.3 Motor Control

ROBOT NAME controls its two drive motors by commanding the SparkMAX controllers over a dedicated CAN bus. The Teensy 4.1 serves as the CAN bridge, translating high-level commands from the main computer (received over USB Serial) into SparkMAX CAN frames.

The SparkMAX controllers use REV Robotics’ proprietary CAN protocol with 29-bit extended identifiers. The CAN ID is structured as follows:

Table 5: SparkMAX CAN ID bit-field layout (29-bit extended).

Field	Bits	Width	Value
Device Type	28:24	5 bits	2 (Motor Controller)
Manufacturer	23:16	8 bits	5 (REV Robotics)
API Class	15:10	6 bits	Varies by command
API Index	9:6	4 bits	Varies by command
Device ID	5:0	6 bits	1–62 (unique per controller)

Key API commands used:

- **Non-RIO Heartbeat** (Class 11, Index 2) — Sent every 20 ms to keep controllers enabled
- **Percent Output** (Class 0, Index 2) — Motor duty cycle from -1.0 to $+1.0$
- **Velocity Setpoint** (Class 1, Index 2) — Target RPM using SparkMAX internal PID
- **Encoder Feedback** (Class 6, Index 2) — Periodic drive encoder position reports

All floating-point values are encoded as IEEE 754 single-precision floats in little-endian byte order. By offloading the PID control loop onto the SparkMAX controllers, we simplify the firmware and reduce end-to-end latency.

5.4 Safety Lights

Safety lights on the vehicle indicate the autonomous state as required by IGVC rules.

Table 6: Safety light color mapping.

Light Color	Meaning
White	Standard operation
Yellow	Waypoint following activated
Green	Waypoint reached
Red	Unable to move forward / error

5.5 Emergency Stop and Safety

Safety is a top priority for the vehicle and is at the core of our electrical design. The emergency stop circuit features three interlocks that must all be disengaged for the motors to receive power:

1. **On-vehicle button** — Physical red mushroom-head e-stop button.
2. **Remote button** — Wireless e-stop with >100 ft range, as required by IGVC rules.
3. **Software interlock** — Software-controlled relay that is only enabled when all nodes report healthy status.

Motor power is gated by a solid-state relay controlled directly by the e-stop line. If any interlock is triggered, power to the motor controllers is immediately cut regardless of software state.

6. Software Systems

6.1 Overview

Our software uses the **Robot Operating System (ROS 2)**, a robotics middleware that is widely used in both academia and industry. It is based on a publisher-subscriber model, with individual processes called nodes sending messages to each other. This allows for much of the software work to be developed in parallel in separate packages.

6.2 Image Transformation

ROBOT NAME uses a classical computer vision approach to transform camera image(s) into an obstacle map that the vehicle can use to navigate.

The images are processed through the following pipeline:

1. **Camera calibration** — Correct lens distortion using a calibration matrix.
2. **Perspective transform** — Project the camera image onto a top-down 2D plane.
3. **Color filtering** — Apply HSV thresholding to detect lane lines and obstacles.
4. **Obstacle map generation** — Combine filtered images into a unified obstacle map.

TODO: Describe any auto-calibration or adaptation to lighting conditions.

6.3 Navigation

TODO: Describe your path planning and navigation algorithm.

The navigation system receives the obstacle map from the perception subsystem and the current position from the localization subsystem. It computes motor commands that guide the vehicle toward the next GPS waypoint while avoiding detected obstacles.

6.4 Localization

Autonomous localization is required for qualifying for and completing the IGVC AutoNav challenge. Waypoint navigation is only possible with accurate and precise location data.

ROBOT NAME uses a particle filter / extended Kalman filter that combines:

- **Wheel odometry** — Encoder feedback from the SparkMAX motor controllers, transmitted through the Teensy 4.1 over CAN bus.
- **GPS data** — TODO: GPS module name and accuracy specifications.
- **IMU data** — TODO: IMU name providing heading and angular velocity.

This fused location data is consumed by the path planner as the robot's current position and is used to determine waypoint arrival and distance from the start of the course.

6.5 Drive Control

The main computer publishes desired linear and angular velocity commands. These are converted to individual left/right wheel speeds using the differential drive kinematic model:

$$v_L = v - \frac{\omega \cdot d}{2} \quad (1)$$

$$v_R = v + \frac{\omega \cdot d}{2} \quad (2)$$

where v is the desired linear velocity, ω is the desired angular velocity, and d is the track width (distance between wheel centers).

The computed wheel speeds are sent to the Teensy 4.1 over USB Serial, which translates them into SparkMAX CAN commands. Encoder feedback from both motors is read by the Teensy and sent back to the main computer for odometry computation.

6.6 Configuration and GUI

TODO: Describe any GUI, configuration system, or parameter tuning tools.

Configuration of tunable parameters is handled via a custom GUI / ROS parameter server running on the main computer. This allows the team to tune the robot on-the-fly without recompiling code.

6.7 State Machine

The robot operates using a state machine that manages transitions between the following system states:

- **Disabled** — Motors off, all subsystems idle.
- **Manual** — Operator control via gamepad/controller for testing.
- **Autonomous** — Full autonomy, running perception + navigation + localization.
- **Emergency Stop** — Immediate motor cutoff, requires manual reset.

Each software node also maintains its own local state (off, warming, ready, operating, error) to ensure all subsystems are healthy before autonomous operation begins.

6.8 Logging

TODO: Describe your data logging strategy for post-run analysis.

After every run, a comprehensive log file is created containing sensor readings (camera feeds, GPS, IMU, encoder feedback), diagnostic metrics, navigation targets, and debug information.

7. Cyber Security Analysis

7.1 NIST RMF Process

Security vulnerabilities in self-driving vehicles pose significant risks. To effectively mitigate these threats, the team follows the NIST Risk Management Framework (RMF), which provides a structured process for identifying, assessing, and managing risks throughout the system's life cycle.

7.2 Vehicle Threats

Table 7: Threat assessment matrix.

Subsystem	Threat Description	Confid.	Integrity	Avail.
GPS Waypoints	GPS waypoint tampering could lead the vehicle to drive in unintended ways.	Low	High	High
Remote E-stop	Malicious interference could disable or trigger the e-stop.	Moderate	High	High
Onboard Computer	Malware or unauthorized access could compromise system control.	High	High	High
Odometry	Sensor data tampering can result in incorrect state estimation.	Low	High	Moderate
CAN Bus	CAN frame injection could send unauthorized motor commands.	Moderate	High	High
Git Repository	Unauthorized access could inject malicious code.	High	High	Moderate

7.3 Cyber Controls and Implementation

Table 8: Security controls.

Subsystem	Control	Implementation
GPS Waypoints	SI-4 System Monitoring	Display GPS waypoints to the user and validate using range checks.
Remote E-stop	AC-3 Access Enforcement	Authenticate and/or encrypt the e-stop radio channel.
Onboard Computer	SC-28 Protection of Information at Rest	Use secure boot, enable SSH authentication, and disable passwords.
Odometry	SI-7 Information Integrity	Validate sensor input using range checks and sensor fusion.
CAN Bus	SC-8 Transmission Confidentiality	Monitor for anomalous CAN traffic; heartbeat watchdog timeout.
Git Repository	CM-5 Access Restrictions for Change	Enforce code access control with role-based permissions.

8. Vehicle Analysis

8.1 Lessons Learned

TODO: Describe the key lessons your team learned during the design and build process.

8.2 Failure Points and Resolutions

Table 9: Failure points and resolutions.

Category	Failure Point	Cause	Resolution
Mechanical	Fasteners loosen	Vibrations from driving	Loctite and Nyloc nuts
Electrical	E-stop connection lost	Remote goes out of range	Heartbeat timeout stops robot
Electrical	CAN bus errors	Wiring / termination issues	Added proper $120\ \Omega$ termination
Software	Obstacle misdetection	Changing weather / lighting	Auto HSV calibration at startup
Software	Sensor data loss	Cable disconnect / node crash	Watchdog monitors node health

8.3 Safety, Reliability, and Durability

ROBOT NAME was designed from the ground up to be safe, reliable, and durable in all conditions. Through dozens of test runs in various scenarios—sensor loss, changing weather conditions, e-stop testing—the team has verified that the robot reliably activates all of its safety measures.

TODO: Describe specific reliability testing you performed.

8.4 Performance Metrics

Table 10: Performance metrics.

Parameter	Requirement	Measured Value
Course Completion Time	<5 minutes	TODO
Obstacle Detection Range	>2 meters	TODO
Obstacle Reaction Time	<250 ms	TODO
Remote E-stop Range	>100 ft	TODO
Navigation Accuracy	<1 meter	TODO
Battery Life	>5 minutes	TODO
Ramp Ascension	Successful	TODO

8.5 Software Testing, Bug Tracking, and Version Control

The software subteam utilizes Git and GitHub to store and version control the codebase. By utilizing branches for each new feature, those features can be isolated and independently tested before being merged into the main codebase.

TODO: Describe your CI/CD pipeline, testing framework, and bug tracking process.

8.6 Simulation

TODO: Describe any simulation environment you use (Gazebo, custom simulator, Unity, etc.).

9. Unique AutoNav and Self Drive Software

ROBOT NAME is competing in AutoNav only / is competing in both AutoNav and Self Drive.

AutoNav and Self Drive share many core features such as lane keeping and obstacle avoidance. However, Self Drive requires additional capabilities such as:

- Lane changing
- Stop sign detection and compliance
- Merging
- Parking

TODO: Describe any unique software features for each mode.

10. Performance Assessment

As of submitting this report, *ROBOT NAME* is on track for competition. Through extensive testing, the drivetrain, vision pipeline, and navigation systems have been validated.

TODO: Summarize overall readiness for competition. Be honest about what is complete and what remains.